svg-animate

Animated SVG diagrams with Ti*k*Z

Sébastien Gross

https://github.com/renard/svg-animate

2026/04/28— v1.1

Contents

1 Introduction **1**

2 Installation **1**

3 Compilation pipeline **2**
3.1 Embedding bitmap images . . . 2

4 Animation **3**
4.1 Overview . . . 3
4.2 Environment . . . 3
4.3 Commands . . . 3
4.4 Keys . . . 5
4.5 Static PDF output . . . 10

5 Example: traffic light **10**

6 Change log **12**

1 Introduction

`svg-animate` is a small LaTeX package for producing **animated SVG diagrams** using Ti*k*Z. The animation model is deliberately simple: content is organised into discrete *steps*, and only one step is fully visible at a time — exactly like a cinema film or a slide presentation.

The package wraps the low-level `tikz animations` library (available since PGF 3.1.9) with a convenient high-level interface that handles step counting, absolute timing, and option cascading automatically.

Output formats:

- **Animated SVG** — compile with `latex` + `dvisvgm`. The animation uses SMIL opacity keyframes supported by all major browsers.
- **Static PDF** — compile with `xelatex` or `pdflatex`. The package detects the output mode and adapts `\reveal` behaviour accordingly (see Section 4.5).

2 Installation

Copy the `svg-animate` directory into a location where TEX can find it. Common user-local locations (no root access required):

- `$TEXMFHOME/tex/latex/`
- `$TEXHOME/tex/latex/`
- `~/texmf/tex/latex/` (typical Linux default)
- `~/Library/texmf/tex/latex/` (typical macOS default)

Alternatively, point `TEXINPUTS` at the directory containing `svg-animate.sty` before compiling:

```
TEXINPUTS="path/to/svg-animate//:$TEXINPUTS" latex diagram.tex
```

The trailing `//` tells TEX to search subdirectories recursively.

3 Compilation pipeline

```
# Compile to animated SVG
latex      diagram.tex
latex      diagram.tex          # second pass for stable layout
dvisvgm --font-format=woff2 \
        --optimize=all      \
        --bbox=min          \
        diagram.dvi -o diagram.svg

# Compile to static PDF (for preview)
xelatex diagram.tex
```

3.1 Embedding bitmap images

If the diagram includes PNG or JPEG images (e.g. via `\includegraphics`), pass `-embed-bitmaps` to `dvisvgm` to produce a fully self-contained SVG. Each bitmap is then inlined as a base64 data URI rather than an external file reference, so the SVG can be moved or published independently of the source directory.

```
dvisvgm --font-format=woff2 --optimize=all --bbox=min \
        --embed-bitmaps diagram.dvi -o diagram.svg
```

Why a driver override is needed. The stock `graphicx` dvisvgm driver (`graphics-def/dvisvgm.def`) implements bitmap inclusion via `\special{dvisvgm:raw}`, injecting a literal `<image xlink:href="file.png"/` fragment. `dvisvgm` copies this verbatim into the output SVG, leaving an external reference that `-embed-bitmaps` does not process.

`svg-animate` therefore redefines `\Ginclude@bitmap` (in SVG mode only) to emit `\special{dvisvgm:img W` instead. This is the special type that `-embed-bitmaps` does process, converting the file to a base64 data URI.

Version requirement. Support for `-embed-bitmaps` on `dvisvgm:img` specials was introduced in **dvisvgm 3.4** (2024-07-24, shipped with TEX Live 2025). With earlier versions, `dvisvgm:img` produces an external `xlink:href` reference regardless of `-embed-bitmaps`: the override has no effect and bitmaps remain as separate files.

The package detects the output mode via `\pdfoutput` (defined by pdfTeX and LuaTeX):

- `\pdfoutput = 0` (**latex** in DVI mode) — loads the **dvisvgm** PGF driver and passes `dvisvgm` to `graphicx`.
- `\pdfoutput = 1` (**pdflatex**, **lualatex**) — default PDF drivers; no action needed.
- `\pdfoutput` undefined (**xelatex**) — default XeTeX drivers; no action needed.

A minimal document looks like:

```
\documentclass{standalone}
\usepackage{svg-animate}

\begin{document}
\begin{tikzpicture}
  % ... your diagram ...
\end{tikzpicture}
\end{document}
```

4 Animation

4.1 Overview

All animation content is placed inside an **animate** environment. The body is divided into *steps* by **\animstep** separators. Each **\reveal** call wraps a TikZ element and controls its opacity: the element is active (fully visible) during its step and inactive (hidden or dimmed) during all other steps.

```
\begin{animate}[duration=1]
  \reveal[inactive opacity=0.2]{\node (A) at (0,0) {Step 1};}
  \animstep
  \reveal{\node (B) at (0,0) {Step 2};}
  \animstep[duration=2]
  \reveal{\node (C) at (0,0) {Step 3 (lasts 2 s)};}
\end{animate}
```

Options cascade from the environment down through each step to individual elements: inner options override outer ones.

```
\begin{animate}[duration=1, inactive opacity=0]
\animstep[inactive opacity=0.3]
\reveal[active opacity=0.8]{...}
\end{animate}
```

4.2 Environment

\begin{animate}[⟨*options*⟩]
  ⟨*environment content*⟩
\end{animate}

Collects all content up to **\end{animate}** and processes it in two passes: first to sum all step durations (without executing any TikZ code), then to render each step with correct absolute timing.

[⟨*options*⟩] accepts any **/anim/.cd** key and applies them as defaults for all steps in this environment.

4.3 Commands

`\reveal`[⟨*options*⟩]{⟨*content*⟩}

Wraps ⟨*content*⟩ in an opacity animation. The element is shown at `active opacity` during its step and at `inactive opacity` at all other times.
Set `inactive opacity` to a value above 0 to *dim* the element instead of hiding it — useful for nodes that should remain faintly visible in the background.
`duration` in [⟨*options*⟩] is accepted but silently ignored (it has no meaning at the per-element level).
PDF mode behaviour depends on whether `\noanimate` is present in the enclosing `animate` body:

- No `\noanimate`: ⟨*content*⟩ is rendered at full opacity (all steps stacked — useful for a quick overview).
- `\noanimate` present: ⟨*content*⟩ is suppressed; use the `noanimate` key on `\reveal` to override for individual elements.

`\noanimate`{⟨*content*⟩}

PDF mode: renders ⟨*content*⟩ as a plain static element, with no animation wrapper. When `\noanimate` is present inside an `animate` body, all `\reveal` elements in that body are automatically suppressed, leaving ⟨*content*⟩ as the sole visible output — a single clean frame.
SVG mode: completely ignored; the animated steps play normally.
`\noanimate` must appear inside `animate`. Multiple calls are allowed; each one renders independently in PDF.

```
\begin{animate}[inactive opacity=0.08]
  \noanimate{\fill[red!85!black] (0,2.2) circle[radius=0.85];}
  \reveal{\fill[green!65!black]  (0,-2.2) circle[radius=0.85];}
  \animstep
  \reveal{\fill[red!85!black]   (0, 2.2) circle[radius=0.85];}
\end{animate}
```

`\animstep`[⟨*options*⟩]

Step separator inside `animate`. The token is never executed; it is consumed as a delimiter when the body is split into per-step segments.
[⟨*options*⟩] accepts any `/anim/.cd` key and applies them to all elements of the *following* step, overriding environment-level defaults.

`\subanim`[⟨*options*⟩]{⟨*body*⟩}

Subdivides the *current* outer step into `steps=`⟨N⟩ equal sub-steps, executes ⟨*body*⟩ with a temporary inner timing table, then restores the outer table. This creates a micro-animation inside one outer step without altering the step structure visible to the rest of the diagram. Inside ⟨*body*⟩:

- `\subanimsteps` expands to ⟨N⟩ useful as the upper bound of a `\foreach` loop.
- `erase=true` is set by default so sub-step elements freeze at `inactive opacity` when their sub-step ends: nothing persists after the group.
- `\reveal`[⟨*step=*⟨k⟩⟩] refers to inner sub-step ⟨k⟩ ($1 \le k \le N$).

Persistent state. To leave a permanent result after the group (e.g. a fully grown arrow), place a `\reveal` *outside* `\subanim`, targeting the outer step that follows the group's window:

```
\begin{animate}[duration=0.3, loop=false]
  %% Outer step 1: line grows AB in 5 sub-steps of 0.06 s each.
  \subanim[steps=5]{
    \foreach \x in {1,...,\subanimsteps} {
      \reveal[step=\x]{%
        \draw[line width=2pt, red, ->]
          (-2,0) -- ($(-2,0)!\x/\subanimsteps!(2,0)$);
      }
    }
  }
  %% step=2 starts immediately after the group ends  full arrow persists.
  \reveal[step=2]{\draw[line width=2pt, red, ->] (-2,0) -- (2,0);}
  \animstep
  %% outer step 2 body  empty; persistent line shown via \reveal[step=2].
\end{animate}
```

Window selection. `\subanim` determines its outer time window in the following order:

1. An explicit `step=`⟨*spec*⟩ passed in `[`⟨*options*⟩`]` always wins.
2. Otherwise, if `\subanim` is nested inside `\reveal[`⟨*step=*⟨*spec*⟩⟩`]`, that step specification is inherited automatically — no need to repeat it.
3. Otherwise, the time window of the step segment in which `\subanim` appears in the source is used.

The inheritance in case 2 means the common pattern of placing `\subanim` directly inside a `\reveal` works without any extra options:

```
\reveal[step=2]{
  \path (A) -- (B) node {label};   %% visible during step 2
  \subanim[steps=5]{               %% inherits step=2 automatically
    \foreach \x in {1,...,\subanimsteps} {
      \reveal[step=\x]{\draw[->] (A) -- ($(A)!\x/\subanimsteps!(B)$);}
    }
  }
}
```

Pass `step=`⟨*A*⟩`-`⟨*B*⟩ explicitly to span a multi-step window:

```
%% Group spans outer steps 1 and 2 (their combined duration).
\subanim[steps=10, step=1-2]{...}
```

PDF mode: all sub-steps are rendered simultaneously at full opacity (equivalent to `static`).
`\subanim` must appear inside an `animate` environment.

4.4 Keys

All keys live under the `/anim/.cd` path and can be set at any level of the cascade.

`/anim/duration=`⟨*seconds*⟩ (initial: 2)

Duration of this step in seconds. Can be set globally, per-environment, or per-step via `\animstep[`⟨*duration=*⟨*N*⟩⟩`]`. Has no effect when passed to `\reveal` (silently ignored).

Setting the first step's duration independently. `\animstep[`⟨*duration=*⟨*N*⟩⟩`]` sets the duration of the *following* step, so there is no preceding `\animstep` to use for the first step. To give step 1 a different duration without changing the global default, write `[duration=`⟨*N*⟩`]` as the very first token in the animate body:

```
\begin{animate}[duration=2]      %% default = 2 s (steps 2, 3, )
  [duration=1]                   %% step 1 = 1 s
  \reveal[step=1]{...}
  \reveal[step=2-3]{...}
  \animstep  \animstep  \animstep
\end{animate}
```

This works because `\animstep[⟨duration=⟨N⟩⟩]` places the token sequence `[duration=⟨N⟩]` at the start of the next segment; writing it literally at the start of the body applies the same mechanism to segment 1.

/anim/steps=⟨*N*⟩ (no default)

Used with `\subanim`: specifies the number of equal inner sub-steps into which the outer step's time window is subdivided. Must be a positive integer ≥ 1. Inside the `\subanim` body the same value is available as `\subanimsteps`.

/anim/active opacity=⟨*value*⟩ (initial: `1`)

Opacity of a `\reveal` element while its step is active. `1` = fully opaque (default), `0` = fully transparent.

/anim/inactive opacity=⟨*value*⟩ (initial: `0`)

Opacity of a `\reveal` element while its step is *inactive*. `0` = fully hidden (default). Set to a small positive value (e.g. `0.15`) to keep elements faintly visible rather than completely disappearing.

/anim/loop=`true|false` (initial: `true`)

Controls whether the animation repeats after reaching the last step.

- `true` (default) the animation loops indefinitely.
- `false` the animation plays once. When finished, SMIL `fill="remove"` (the browser default) takes effect: each `\reveal` element reverts to its base opacity (1 = fully visible). See the **erase** key to suppress all elements at the end instead.

Can be set globally or per-environment.

```
%% One-shot animation  plays through all steps once, then stops.
\begin{animate}[loop=false]
  \reveal{\node {Step 1};}
  \animstep
  \reveal{\node {Step 2};}
  \animstep
  \reveal{\node {Step 3};}
\end{animate}
```

End-of-animation resting state. When the animation ends (`loop=false`), SMIL `fill="remove"` causes all `\reveal` elements to revert to base opacity 1 (fully visible). Two patterns are available to control what the diagram looks like after the animation:

- **`\noanimate` fallback** place a static element with `\noanimate` in the body. It is ignored while the animation plays, but becomes visible (at full opacity) once all `<animate>` effects are removed.
- **erase key** mark each transient `\reveal` with `erase`: those elements freeze at opacity 0 when the animation ends. A final `\reveal` without `erase` reverts to opacity 1 and stays visible.

```
\begin{animate}[inactive opacity=0.08, loop=false]
  %% Static resting state shown after the animation ends (fill="remove"
  %% makes it visible once the <animate> effects are removed).
  \noanimate{\fill[red!85!black] (0,2.2) circle[radius=0.85];}
  \reveal{\fill[green!65!black]  (0,-2.2) circle[radius=0.85];}
  \animstep
  \reveal{\fill[orange!90!black] (0, 0.0) circle[radius=0.85];}
  \animstep
  \reveal{\fill[red!85!black]    (0, 2.2) circle[radius=0.85];}
\end{animate}
```

`/anim/erase`(no value) (default: not set)

When set together with `loop=false`, appends the TikZ modifier `freeze` to every affected keyframe list, which produces SMIL `fill="freeze"` in the SVG output. Because every keyframe sequence already ends at `inactive opacity` (0 by default), each element freezes at opacity 0 when the one-shot animation finishes — the diagram returns to a clean, empty state.

Has no effect when `loop=true` (repeating animations have no end state).

Two usage levels:

- **Environment level** — `\begin{animate}[loop=false, erase]` — all `\reveal` elements in the body disappear when the animation ends.
- **Per-element level** — `\reveal[erase]{...}` — only that element disappears; `\reveal` elements *without* `erase` revert to base opacity 1 (visible) via the default `fill="remove"`.

The per-element form enables a common pattern for one-shot animations: transient steps carry `erase` and vanish cleanly; a final `\reveal` without `erase` remains visible as the permanent end state.

```
%% Growing arrow: steps 1-10 show a growing line with a moving arrowhead
%% (erase  disappear at end); step 11 is the final full arrow that stays.
\begin{animate}[duration=0.1, loop=false]
  \foreach \x in {1,...,10} {
    \reveal[step=\x, erase]{%
      \draw[arrow] (0,0) -- ($ (0,0)!\x/10!(5,5) $);
    }
  }
  \reveal[step=11]{\draw[arrow] (0,0) -- (5,5);}
  \animstep\animstep\animstep\animstep\animstep
  \animstep\animstep\animstep\animstep\animstep
\end{animate}
```

Why `erase` rather than `freeze`. The TikZ modifier that produces `fill="freeze"` is named `freeze` internally (and `forever` is its alias). The name `freeze` was deliberately *not* exposed at the `/anim/` key level because it is easily confused with `loop=false` ("freezing the loop"). The key `erase` describes the *visible effect* — elements are erased from the diagram when the animation ends — rather than the SMIL mechanism.

`/anim/debug`(no value) (default: not set)

When set on the `animate` environment, draws a small overlay node *after* every step's content, so it is always rendered on top of all other elements. The node shows the current step, the total number of steps, and the step duration:

`k/N [Ds]`

where `k` is the 1-based step index, `N` the total number of steps, and `D` the duration in seconds. This is useful during authoring to verify step numbering, timing, and `\reveal` visibility.

SVG mode only — in PDF mode (LuaLaTeX, pdfLaTeX) the key is accepted but has no visible effect, because SMIL animations are not generated and the node would always be visible at full opacity.
Has **no effect** when `static` is also set (the whole environment renders all steps simultaneously without per-step animation).

```
\begin{tikzpicture}
  \begin{animate}[duration=1, debug]
    \reveal{\node at (0,0) {Step 1};}
    \animstep
    \reveal{\node at (0,-1) {Step 2};}
  \end{animate}
\end{tikzpicture}
```

`/anim/debug pos`=⟨*coordinate*⟩ (initial: `current bounding box.south east`)

TikZ coordinate where the debug overlay node is anchored. Any TikZ coordinate expression is accepted.

```
%% Place the debug label at the top-left instead.
\begin{animate}[duration=1, debug, debug pos=current bounding box.north west,
                debug style={anchor=north west, fill=cyan}]
  ...
\end{animate}
```

`/anim/debug style`=⟨*TikZ options*⟩ (initial: yellow box, red border)

TikZ node options applied to the debug overlay node. Overrides the entire default style; use `debug pos` separately to change only the position.
The package default style is:

```
anchor=south east, font=\small\ttfamily,
draw=red!80!black, fill=yellow!80, inner sep=2pt, rounded corners=1pt
```

The value is reset to the default at the start of each new `animate` environment, so custom styles do not bleed across environments.

`/anim/blink`=⟨*seconds*⟩ (no default)

Makes the element blink during its active step: the opacity alternates between `blink on opacity` and `blink off opacity` at the given period. The element is shown at `inactive opacity` during all other steps.
The ⟨*seconds*⟩ value is the *full* blink period: the element is visible for ⟨*seconds*⟩/2 seconds, then hidden for ⟨*seconds*⟩/2 seconds, then repeats.
If the step duration is not an exact multiple of ⟨*seconds*⟩/2, the last partial half-period is included and cut at the step boundary.
Note: `blink` and `step` are mutually exclusive. When both are specified, `blink` wins, `step` is ignored, and a `PackageWarning` is issued.

```
\begin{animate}[duration=2, inactive opacity=0.15]
  \reveal{\node at (0,1) {Step 1  static};}
  \animstep
  \reveal[blink=0.4]{\node[fill=yellow] at (0,0) {Step 2  blinks};}
  \animstep
  \reveal{\node at (0,-1) {Step 3  static};}
\end{animate}
```

/anim/blink on opacity=⟨*value*⟩ (initial: 1)

Opacity used during the *"on"* (visible) half-periods of a blink. Defaults to 1 (fully opaque). Independent of **active opacity**.

/anim/blink off opacity=⟨*value*⟩ (initial: 0)

Opacity used during the *"off"* half-periods of the blink oscillation *within the active step*. Defaults to 0 (fully hidden).
Between steps the element uses **inactive opacity** as usual **blink off opacity** has no effect outside the active step. To hide a blink element between steps, set **inactive opacity=0** explicitly; the **static** key on the **animate** environment ensures this does not suppress the element in the static display:

```
%% Hidden between steps; blinks between 1 (on) and 0.25 (off) during step 2.
\begin{animate}[duration=2]
  \reveal{\node at (0,1) {Step 1};}
  \animstep
  \reveal[blink=0.4, inactive opacity=0, blink off opacity=0.25]{%
    \node[fill=yellow] at (0,0) {Step 2  dimmed blink};}
  \animstep
  \reveal{\node at (0,-1) {Step 3};}
\end{animate}
```

/anim/static(no value) (default: not set)

Renders all **\reveal** elements at their natural (full) opacity without emitting any SMIL keyframe animation. Equivalent to drawing all steps simultaneously, which is useful for printed handouts and static overviews.
This key supersedes the older idiom of setting **active opacity=1, inactive opacity=1, blink on opacity=1, blink off opacity=1** at the environment level, which failed whenever an element carried an explicit element-level override.

```
\begin{animate}[static]
  \stepOne \animstep \stepTwo \animstep \stepThree
\end{animate}
```

/anim/step=⟨*spec*⟩ (no default)

By default **\reveal** makes an element visible only during the *current* step (the step in which it appears in the source). The **step** key overrides this: the element becomes visible during every step listed in ⟨*spec*⟩, regardless of its position in the source.

Syntax of ⟨*spec*⟩:

- Single step **step=2**
- Range **step=2-5** (steps 2, 3, 4, 5)
- List **step={1,3}** (braces required when listing multiple values)
- Mix **step={1,3-5}** (braces required)

Consecutive steps in the specification are automatically merged into a single animation window (no redundant keyframe at the shared boundary).

```
\begin{animate}[inactive opacity=0.08]
  \reveal[step={1,3}]{\node[red]  at (4,1) {visible in steps 1 and 3};}
  \reveal[step=2-4]{  \node[blue] at (4,0) {visible in steps 2 through 4};}
  \reveal[step={1,3-5}]{\node    at (4,-1) {steps 1, 3, 4, 5};}
\end{animate}
```

`/anim/noanimate` (no value) (default: not set)

When passed to `\reveal`, forces that element to render in PDF mode even when `\noanimate` is present in the enclosing `animate` body. Has no effect in SVG mode. Useful for elements that should appear in both the animated SVG *and* the static PDF fallback — for example a transition light that belongs to multiple steps and should also be visible in the PDF snapshot.

```
\begin{animate}[inactive opacity=0.08]
  \noanimate{\fill[red!85!black] (0,2.2) circle[radius=0.85];}
  \reveal{\fill[green!65!black]  (0,-2.2) circle[radius=0.85];}
  \animstep
  %% amber is both animated and shown in the PDF static frame
  \reveal[noanimate]{\fill[orange!90!black] (0,0.0) circle[radius=0.85];}
\end{animate}
```

4.5 Static PDF output

The package exposes the boolean flag `\if@anim@svgmode`, which is `true` only when compiling with `latex` in DVI mode (the SVG pipeline). `\reveal` and `\noanimate` both query this flag; the logic is summarised below.

	SVG	PDF
`\reveal{...}` (no `\noanimate` in body)	animated	rendered
`\reveal{...}` (`\noanimate` present)	animated	suppressed
`\reveal[noanimate]{...}`	animated	rendered
`\noanimate{...}`	ignored	rendered

The `animate` environment scans its collected body for the token `\noanimate` before executing any TikZ code, so the suppression decision is made once per environment instance and does not require two separate document structures.

5 Example: traffic light

The diagram animates a traffic light through four steps. The housing and lens covers are static (always visible at full opacity). The lights are revealed one at a time via `\reveal`; when inactive their opacity drops to `0.08`, simulating an unlit bulb seen through dark glass.

The sequence demonstrates two features: per-step duration (step 2 lasts only `0.5 s`) and multiple `\reveal` calls in a single step (step 2 shows green and amber simultaneously as a transition):

Step	Lights	Duration	Meaning
1	green	1.5 s	go
2	green + amber	0.5 s	prepare to stop
3	amber	1.0 s	caution
4	red	1.5 s	stop

The box below shows the complete `svg-animate-example.tex` source on the left and the compiled PDF on the right. The `\noanimate` call selects the red light as the static PDF frame; open `svg-animate-example.svg` in a browser for the full animation.

```
\documentclass{standalone}
\usepackage{svg-animate}

\begin{document}
\begin{tikzpicture}

  \tikzset{/anim/duration=1.5}

  %% Housing (static  always visible)
  \fill[black!80, rounded corners=8pt]  (-1.4, -4.0)
    rectangle (1.4, 4.0);
  \fill[black!50, rounded corners=6pt]  (-1.2, -3.8)
    rectangle (1.2, 3.8);

  %% Lens covers (static dark glass behind each bulb)
  \foreach \y in {2.2, 0, -2.2} {
    \fill[black!60] (0, \y) circle[radius=0.95];
  }


  %% The three lights, one per animation step.
  %% When inactive, opacity=0.08 simulates an unlit bulb
    seen through dark glass.
  \begin{animate}[inactive opacity=0.08]
    %% Static PDF fallback: show only the red light (one
    clean frame).
    %% Ignored in SVG mode; in PDF mode \reveal draws
    nothing so only this appears.
    \noanimate{\fill[red!85!black] (0, 2.2)
    circle[radius=0.85];}

    %% Step 1 -- green (1.5 s)
    \reveal{\fill[green!65!black]  (0, -2.2)
    circle[radius=0.85];}

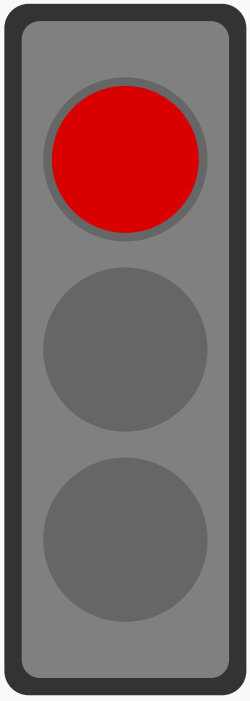
    \animstep[duration=0.5]
    %% Step 2 -- green+amber transition (0.5 s)
    \reveal{\fill[green!65!black]  (0, -2.2)
    circle[radius=0.85];}
    \reveal[blink=0.2]{\fill[orange!90!black] (0,  0.0)
    circle[radius=0.85];}

    \animstep[duration=1.0]
    %% Step 3 -- amber (1.0 s)
    \reveal{\fill[orange!90!black] (0,  0.0)
    circle[radius=0.85];}

    \animstep
    %% Step 4 -- red (1.5 s)
    \reveal{\fill[red!85!black]    (0,  2.2)
    circle[radius=0.85];}

  \end{animate}

\end{tikzpicture}
\end{document}
```



6 Change log

vX.X (YYYY/MM/DD)

- Added `\subanim[⟨steps=⟨N⟩⟩]{⟨body⟩}`: subdivides the current outer step into ⟨*N*⟩ equal inner sub-steps, enabling micro-animations (e.g. a growing line) within a single outer step without altering the surrounding step structure. Inside the body, `\subanimsteps` expands to ⟨*N*⟩ and `erase=true` is set by default so nothing persists after the group.
- Added `steps` key (`/anim/.cd`) for use with `\subanim`.
- Documented the `[duration=⟨N⟩]` body-prefix technique for setting the *first* step's duration independently of the global default (no preceding `\animstep` is available for step 1).
- Fixed `\Ginclude@bitmap` in LuaLaTeX: the path passed to the `dvisvgm:img` special is now resolved to an absolute path via `kpse.find_file` + `lfs.currentdir()` at compile time. Previously, `dvisvgm -embed-bitmaps` would locate the file via kpathsea (no warning) but fail to open it for base64 encoding if its working directory differed from the LaTeX working directory, producing an empty data URI.
- Added `debug` key (`/anim/.cd`). When set on the `animate` environment, draws a per-step overlay node after each step's content (always on top) showing `k/N [Ds]`: the 1-based step index, total step count, and duration in seconds. SVG mode only. Companion keys `debug pos` and `debug style` control the node's anchor coordinate and TikZ styling.
- Added `erase` key (`/anim/.cd`). When used with `loop=false`, appends the TikZ `freeze` modifier (SMIL `fill="freeze"`) to the keyframe list, causing elements to freeze at `inactive opacity` (0) when the one-shot animation ends. Can be set at environment level (all elements erased) or per-`\reveal` (selective: unmarked elements revert to opacity 1 and stay visible).
- Clarified `loop=false` documentation: SMIL `fill="remove"` is the browser default when no modifier is emitted; elements revert to base opacity 1 after animation. Added the two resting-state patterns (`\noanimate` fallback vs `erase`).

v1.1 (2026/04/28)

- Added installation instructions (user-local `texmf` paths and `TEXINPUTS`).
- Rewrote the `\Ginclude@bitmap` override to detect the `dvisvgm` version at compile time: the override is now only applied with `dvisvgm` ≥ 3.4, which is the version that added `-embed-bitmaps` support for `dvisvgm:img` specials.
- Renamed `example.tex` and `test.tex` to `svg-animate-example.tex` and `svg-animate-test.tex` to avoid filename collisions on shared TEX search paths (CTAN requirement).

v1.0 (2026/03/18)

- Initial CTAN release.